Blog Platform: Complete Project Plan

FalconXoft Weekend 1 Assignment | Project 1 of 3 | Stack: Django + Next.js + PostgreSQL

This document explains everything you must plan, build, test and submit for the Blog project, based on the assignment PDF. It is written in simple English so you can also use it to prepare for the Monday interview.

1. What the PDF asks for (Blog project)

Minimum functional scope: authentication, author and admin roles, create and edit posts, draft and publish, categories, comments and search.

Key rules from the PDF:

- Build a working MVP with one complete end-to-end user journey.
- Never say something is done if it is not done. Write honestly what is missing and why.
- Every decision must be defensible in the interview. The reviewer can ask for a walkthrough, debugging or a small code change.
- AI use is not encouraged. If you use it, you must disclose the tool, the areas, and how you verified the result.
- No passwords, keys or secrets in the repo or the Drive folder.

Scoring (per project): Problem understanding 20%, Architecture and design 25%, Implementation 25%, Testing 20%, Explanation and ownership 10%.

2. Requirements and user flow

2.1 Business goal

A simple blogging website where authors write and publish articles, readers read and comment, and admins keep the site clean and safe.

2.2 Users and roles

Role	Who they are	What they can do
Visitor (not logged in)	Anyone on the internet	Read published posts, search, filter by category, register, log in

Role	Who they are	What they can do
Reader (logged in, default role)	Registered user	Everything a visitor can do, plus write comments and edit or delete their own comments
Author	User with the author role	Everything a reader can do, plus create, edit, delete their own posts, save drafts, publish and unpublish their own posts
Admin	Site manager	Everything an author can do, plus manage all posts, all comments, categories and users (change roles, deactivate)

A reader can become an author when an admin changes their role. (Assumption: authors are approved by admin. This avoids spam.)

2.3 Scope

In scope (must build):

1. Register, login, logout, token refresh
2. Roles: reader, author, admin
3. Post create, edit, delete
4. Draft and publish (plus unpublish)
5. Categories (admin creates, authors pick)
6. Comments (add, edit own, delete own, admin delete any)
7. Search by title and content, with category filter and pagination

Out of scope (say this honestly in README):

- Email verification and password reset by email (needs an email service)
- Image upload to cloud storage (use a simple image URL field, or local upload only if time allows)
- Rich text editor with advanced features (use plain text or Markdown)
- Likes, followers, notifications, social login

If you skip something, write it in the "Known gaps" section of the README with the reason.

2.4 Assumptions

- One language (English), one time zone stored as UTC.
- Comments are shown immediately (no moderation queue), but admin can delete them.
- Post body is stored as Markdown text.
- Each post has exactly one category (simple). Tags are out of scope.
- Search uses PostgreSQL full-text search (no extra search server).

2.5 Main user journey (end to end)

This is the one journey you must demo completely:

1. A new user opens the site and registers.
2. Admin promotes the user to Author.
3. Author logs in and creates a post in Draft status with a category.
4. Author edits the draft, then clicks Publish.
5. A visitor opens the home page and sees the post in the list.
6. The visitor searches for a word from the post and finds it.
7. A reader logs in, opens the post and writes a comment.
8. Admin deletes an inappropriate comment.

2.6 Acceptance criteria

ID	Criteria
AC1	A user can register with a unique email and a strong password, and can log in
AC2	Draft posts are visible only to their author and admin
AC3	Published posts are visible to everyone
AC4	An author cannot edit or delete another author's post
AC5	Only admin can create categories and change user roles
AC6	Only logged-in users can comment; users can delete only their own comments

ID	Criteria
AC7	Search returns only published posts that match the keyword
AC8	Invalid input returns a clear error message and the correct HTTP status code
AC9	The project runs from the README steps on a clean machine

3. Technology stack

Layer	Choice	Why (simple reason)
Backend	Django + Django REST Framework (DRF)	Built-in auth, admin, ORM, migrations, and strong security defaults
Frontend	Next.js (React)	Server-side rendering gives better SEO for blog pages, file-based routing
Database	PostgreSQL	Reliable, supports constraints, indexes and full-text search
Auth	JWT (access and refresh tokens) using djangorestframework-simplejwt	Works well with a separate frontend
Containers	Docker + docker-compose	Same setup on every machine; easy to demo
Tests	pytest + pytest-django (backend), optional Jest or Playwright (frontend)	Simple, widely used
CI/CD	GitHub Actions (run tests on every push)	Free for small projects
API docs	openapi.yaml generated with drf-spectacular	Satisfies the api-spec requirement

Alternatives you should be able to explain in the interview:

- *Session auth instead of JWT:* simpler and safer for same-domain apps, but a separate Next.js frontend makes JWT (or cookie-based JWT) more natural.
- *MongoDB instead of PostgreSQL:* blog data is relational (users, posts, comments, categories), so SQL fits better.
- *Express instead of Django:* you chose Django for faster development with built-in security and admin.
- *Elasticsearch for search:* too heavy for an MVP. PostgreSQL full-text search is enough up to a large size.

4. Architecture

4.1 Components

1. **Browser:** the user.
2. **Next.js frontend:** renders pages, calls the API.
3. **Django REST API:** business logic, auth, permissions, validation.
4. **PostgreSQL:** stores all data.
5. **Redis (optional or proposed):** cache for popular post lists and rate limiting.
6. **Background worker (proposed):** Celery for sending emails later.
7. **Reverse proxy (Nginx):** HTTPS, routing between frontend and API.

4.2 Data movement

- Browser sends a request to Nginx.
- Nginx sends page requests to Next.js and `/api/` requests to Django.
- Next.js (server side) or the browser calls Django with a JWT in the header.
- Django checks permissions, reads or writes PostgreSQL, and returns JSON.

4.3 Deployment boundaries

Public zone: Nginx only. Private zone: Next.js, Django, PostgreSQL, Redis (not open to the internet). Only ports 80 and 443 are public.

4.4 Implemented vs proposed

The PDF wants you to separate these clearly.

Item	Status
Django API, PostgreSQL, Next.js, JWT auth	Implemented
Docker and docker-compose	Implemented
GitHub Actions tests	Implemented (if you finish it)
Nginx, HTTPS	Proposed (in deployment plan)
Redis cache	Proposed
Celery workers, email service	Proposed
Monitoring (Sentry, uptime checks)	Proposed

For the architecture.pdf, draw this in Lucidchart with labels on every arrow and show the public and private zones as boxes.

5. Folder structure

5.1 Code repository (one GitHub repo)

```
blog-platform/
├── README.md
├── docker-compose.yml
├── .env.example              (variable names only, no secrets)
├── .gitignore
├── .github/workflows/ci.yml
├── backend/
│   ├── Dockerfile
│   ├── manage.py
│   ├── requirements.txt
│   ├── config/               (settings, urls, wsgi)
│   ├── apps/
│   │   ├── users/            (models, serializers, views, permissions)
│   │   ├── blog/             (Post, Category models and API)
│   │   └── comments/         (Comment model and API)
│   └── tests/
├── frontend/
│   ├── Dockerfile
│   ├── package.json
```

```
|   ├── app/ (or pages/)      (home, post/[slug], login, register, dashboard,
admin)
|   ├── components/
|   └── lib/api.js            (API helper)
└── docs/
    ├── api-spec/openapi.yaml
    └── schema.sql
```

5.2 Drive submission folder

```
Weekend1_YourName/
├── submission-index.md        (item 10, at parent level)
├── 01_Blog/
|   ├── 01-README.md
|   ├── 02-architecture.pdf
|   ├── 03-database-design.pdf  (+ schema.sql)
|   ├── 04-api-workflows.pdf    (+ api-spec.md or openapi.yaml)
|   ├── 05-test-cases.xlsx
|   ├── 06-engineering-decisions.md
|   ├── 07-source-and-demo-links.md
|   ├── 08-cost-and-scaling.xlsx
|   └── 09-sources-and-disclosure.md
├── 02_Ecommerce/
└── 03_Marketplace/
```

6. Database design

6.1 Tables

users

Field	Type	Notes
id	BIGSERIAL	Primary key
email	VARCHAR(254)	Unique, not null
username	VARCHAR(50)	Unique, not null
password_hash	VARCHAR(255)	Django hashed password
role	VARCHAR(10)	Check: reader, author, admin. Default reader

Field	Type	Notes
is_active	BOOLEAN	Default true
created_at	TIMESTAMPTZ	Default now

categories

Field	Type	Notes
id	BIGSERIAL	Primary key
name	VARCHAR(80)	Unique, not null
slug	VARCHAR(100)	Unique, not null

posts

Field	Type	Notes
id	BIGSERIAL	Primary key
author_id	BIGINT	FK to users.id, ON DELETE PROTECT or CASCADE (decide and explain)
category_id	BIGINT	FK to categories.id, ON DELETE SET NULL or PROTECT
title	VARCHAR(200)	Not null
slug	VARCHAR(220)	Unique, not null
body	TEXT	Markdown, not null
status	VARCHAR(10)	Check: draft, published. Default draft
published_at	TIMESTAMPTZ	Null until published
created_at, updated_at	TIMESTAMPTZ	Auto
search_vector	TSVECTOR	For full-text search

comments

Field	Type	Notes
id	BIGSERIAL	Primary key
post_id	BIGINT	FK to posts.id, ON DELETE CASCADE
user_id	BIGINT	FK to users.id
body	TEXT	Not null, length 1 to 1000
created_at, updated_at	TIMESTAMPTZ	Auto

6.2 Relationships (cardinality)

- One user has many posts (1 to many).
- One category has many posts (1 to many).
- One post has many comments (1 to many).
- One user has many comments (1 to many).

6.3 Indexes

Index	Why
posts(slug) unique	Fast post page lookup
posts(status, published_at DESC)	Home page list of latest published posts
posts(author_id)	Author dashboard
posts(category_id)	Category filter
GIN index on posts(search_vector)	Fast full-text search
comments(post_id, created_at)	Load comments of a post
users(email) unique	Login lookup

6.4 Data integrity points

- Unique constraints on email, username, slug, category name.
- CHECK constraints on role and status.
- Foreign keys always enforced by PostgreSQL.

- Wrap publish and similar multi-step changes in a database transaction.
- Decide what happens when a user is deleted. Simple answer: deactivate users instead of deleting them.

Deliverables for this part: ERD in Lucidchart (with PK, FK, types, cardinality, indexes), exported to database-design.pdf, plus `schema.sql` (or Django migrations).

7. API design

Base URL: `/api/v1/`. Format: JSON. Auth header: `Authorization: Bearer <access_token>`.

7.1 Routes

Method	Route	Auth	Purpose
POST	/auth/register/	Public	Create account
POST	/auth/login/	Public	Get access and refresh tokens
POST	/auth/refresh/	Public (refresh token)	Get new access token
GET	/auth/me/	Logged in	Current user profile
GET	/posts/	Public	List published posts (pagination, ?search=, ?category=)
GET	/posts/{slug}/	Public (draft: owner or admin)	Post detail
POST	/posts/	Author, Admin	Create post (draft by default)
PATCH	/posts/{slug}/	Owner or Admin	Edit post
DELETE	/posts/{slug}/	Owner or Admin	Delete post
POST	/posts/{slug}/publish/	Owner or Admin	Draft to published
POST	/posts/{slug}/unpublish/	Owner or Admin	Published to draft

Method	Route	Auth	Purpose
GET	/me/posts/	Author	My posts (all statuses)
GET	/categories/	Public	List categories
POST, PATCH, DELETE	/categories/ and /categories/{id}/	Admin	Manage categories
GET	/posts/{slug}/comments/	Public	List comments
POST	/posts/{slug}/comments/	Logged in	Add comment
PATCH, DELETE	/comments/{id}/	Owner (delete: owner or Admin)	Edit or remove comment
GET	/admin/users/	Admin	List users
PATCH	/admin/users/{id}/	Admin	Change role or active status

7.2 Example: create a post

Request:

```
POST /api/v1/posts/
Authorization: Bearer <token>
{
  "title": "My First Post",
  "body": "Hello world...",
  "category_id": 2
}
```

Success response (201):

```
{
  "id": 15,
  "slug": "my-first-post",
  "title": "My First Post",
  "status": "draft",
  "author": {"id": 4, "username": "ali"},
  "created_at": "2026-10-18T10:00:00Z"
}
```

7.3 Error cases to design and test

Status	When	Example body
400	Validation failed (empty title, title too long)	{"title": ["This field may not be blank."]}
401	Missing or expired token	{"detail": "Authentication credentials were not provided."}
403	Wrong role (reader tries to create a post)	{"detail": "You do not have permission."}
404	Post not found, or draft of another user (hide existence)	{"detail": "Not found."}
409 or 400	Duplicate email, duplicate slug	{"email": ["A user with this email already exists."]}
429	Too many login attempts (proposed throttling)	{"detail": "Request was throttled."}

7.4 Sequence diagrams to draw (04-api-workflows.pdf)

1. **Success path:** Author logs in, creates draft, publishes, visitor reads it. Show Browser, Next.js, Django, PostgreSQL.
2. **Failure path:** Reader tries to publish a post. Django checks the token (valid), checks the role (fails), returns 403. Also show an expired token returning 401 and refresh.

Also provide `openapi.yaml` (auto-generated by drf-spectacular, then checked by you) or a written `api-spec.md`.

8. Frontend pages (Next.js)

Page	Route	Who sees it
Home (latest posts, search bar, category filter)	/	Everyone
Post detail with comments	/posts/[slug]	Everyone
Login and Register	/login, /register	Visitors

Page	Route	Who sees it
Author dashboard (my posts, status)	/dashboard	Author, Admin
Post editor (create and edit)	/dashboard/posts/new, /dashboard/posts/[slug]/edit	Author, Admin
Admin panel (users, categories, comments)	/admin	Admin

Frontend rules: protect dashboard and admin routes (redirect to login), show loading and error states, never trust the frontend for security (the backend always checks permissions), store tokens safely (prefer httpOnly cookies; if you use localStorage, explain the risk in engineering-decisions.md).

9. Implementation plan (build order)

1. Project setup: Docker, PostgreSQL, Django, Next.js, .env.example.
2. Users app: custom user model with role, register, login, JWT, permissions classes.
3. Categories and Posts: models, migrations, CRUD, draft and publish, slug generation.
4. Comments: model, endpoints, permissions.
5. Search: PostgreSQL full-text search with search_vector and GIN index, pagination.
6. Frontend pages in this order: login/register, home and post detail, dashboard and editor, comments, admin panel.
7. Tests (written along the way, not only at the end).
8. README, diagrams, cost sheet, decisions, disclosure.
9. Record the 3 to 5 minute demo.

Important: build backend first, test it with Postman or pytest, and only then connect the frontend. This makes debugging easier.

Environment variable names (no values): DJANGO_SECRET_KEY, DJANGO_DEBUG, DJANGO_ALLOWED_HOSTS, DATABASE_URL (or POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_HOST, POSTGRES_PORT), JWT_ACCESS_LIFETIME_MINUTES, JWT_REFRESH_LIFETIME_DAYS, CORS_ALLOWED_ORIGINS, NEXT_PUBLIC_API_URL.

10. Testing plan (05-test-cases.xlsx)

Required columns: Test ID, Feature, Preconditions, Steps, Input, Expected Result, Actual Result, Status, Defect ID, Evidence Link.

You must run every test and fill in the Actual Result honestly. Add screenshots or pytest output as evidence. If a test fails, write the defect ID and either fix it or document it.

Type	Example tests
Positive	Register, login, create draft, publish, comment, search finds the post
Negative	Login with wrong password, create post with empty title, comment on a missing post
Boundary	Title with 200 and 201 characters, comment with 1000 and 1001 characters, page size limits, empty search
Permissions	Reader creates post (403), author edits another author's post (403), visitor opens a draft (404), non-admin creates category (403)
Duplicate and retry	Register the same email twice, publish an already published post, double-click on submit, duplicate slug handling, same request retried
Regression	After every bug fix, re-run the full suite; add a test for the bug that was fixed
Data integrity	Delete a post and confirm its comments are removed; invalid role value rejected

Aim for 30 to 40 well-written test cases. Quality and honest results matter more than a huge number.

11. Deployment and cost analysis (08-cost-and-scaling.xlsx)

11.1 Two hosting options to compare

Option A: VPS (for example Hostinger or similar)

- One VPS running Docker: Nginx, Next.js, Django (gunicorn), PostgreSQL on the same machine or a second small one.
- Cheap and simple, but you manage backups, updates and security yourself.

Option B: AWS

- EC2 or ECS for app containers, RDS for PostgreSQL, S3 for media and backups, CloudFront for CDN, Route 53 for DNS, CloudWatch for monitoring.
- More expensive and more complex, but easier to scale and more managed.

11.2 Cost lines to include for each option

Compute, database, storage, backups, bandwidth and egress, monitoring, domain name, third-party services (email, error tracking), and worker costs.

11.3 How to fill the spreadsheet

One table per option with these columns: Service, Specification, Region, Usage assumption, Unit price, Monthly estimate, Source URL, Access date. Add totals, then scenario totals for 100, 1,000, 10,000 and 1,000,000 users.

Very important rules:

- Open the official pricing pages yourself and copy the real price, the source URL and the date you checked.
- Mark each number as **Verified** (from the pricing page) or **Assumption** (your estimate).
- State the region (for example, a region close to Pakistan users) and the currency (USD or PKR, with the exchange rate you used).
- Do not invent prices. If you cannot verify a number, label it as an assumption.

11.4 Docker, CI/CD, queues, caches, security, observability

Explain each one in engineering-decisions.md and mark it as implemented or proposed:

Topic	What to explain
Docker	Dockerfiles for backend and frontend, docker-compose for local run
CI/CD	GitHub Actions: run tests and lint on push; proposed: auto-deploy on merge to main

Topic	What to explain
Queues	Celery plus Redis for emails and slow jobs (proposed)
Caches	Redis cache for the home page and post list (proposed)
Security	Password hashing, JWT expiry, CORS limits, HTTPS, rate limiting, input validation, SQL injection protection via ORM, XSS protection when showing Markdown, secrets only in env variables
Observability	Logging (implemented), Sentry for errors and uptime checks (proposed), database backup monitoring

12. Scaling plan

State your traffic assumptions clearly. Example assumptions (change them if you wish): about 10 percent of users are active per day, each active user makes around 20 requests per day, 90 percent of traffic is reading.

Users	What changes	Main bottleneck	Rough approach
100	Nothing special	None	One small VPS, one database, daily backup
1,000	Add caching for the home page; add DB indexes (already planned)	Slow list queries	One stronger VPS, optional Redis
10,000	Separate the database from the app; add a CDN for static files; run several app containers behind a load balancer; move images to object storage	Database reads and CPU	App servers x2, managed DB, Redis cache, read replica if needed

Users	What changes	Main bottleneck	Rough approach
1,000,000	Autoscaling app containers, database read replicas and connection pooling, queue workers, CDN for pages, move search to a dedicated search engine, partition or archive old comments, multi-zone setup, full monitoring	Database size, search, comment writes	Managed cloud (for example AWS), cost grows heavily, so caching and CDN are the main money savers

Explain for each level: database changes, storage changes, queue changes and an approximate monthly cost taken from your spreadsheet.

13. Engineering decisions (06-engineering-decisions.md)

For each major choice, write this mini format: Options considered, Final choice, Why, Trade-offs.

Suggested topics: JWT vs sessions, token storage, PostgreSQL vs NoSQL, full-text search vs Elasticsearch, Markdown vs rich text editor, hard delete vs soft delete, one repo vs two repos, Docker use, hosting choice.

Also include these sections:

- **Security:** what you protected and how.
- **Technical risks:** for example, token theft, large comments, slow search at scale.
- **Implemented vs planned services:** the table from section 4.4.
- **Known gaps and remediation:** every missing feature with the reason and how you would fix it.

14. Open-source and AI disclosure (09-sources-and-disclosure.md)

- **Open source:** list every library and any starter template or repo you copied from, with URL, license, what you reused, what you changed and what you wrote yourself. Normal packages (Django, DRF, simplejwt, Next.js) should be listed with their licenses.

- **AI use:** the PDF says AI is not encouraged. If you used any AI tool, write the tool name, which parts it helped with, and how you checked the result. If you used none, write "None".
- **Pricing citations:** list every pricing page URL with the access date.
- Be honest. You will be asked about every part of your code in the interview, so only submit what you fully understand.

15. README outline (01-README.md)

1. Project name and purpose
2. Tech stack
3. Features and scope (done and not done)
4. Assumptions
5. Architecture summary (with a small diagram image)
6. Setup: clone, copy .env.example to .env, run `docker compose up`, run migrations, create admin user
7. How to run tests
8. Demo accounts description (no real credentials; say how to create them)
9. Known gaps
10. Links: Lucidchart, demo video, Drive folder

16. Demo video script (3 to 5 minutes)

1. 20 seconds: project intro and stack.
2. 40 seconds: architecture and database diagram.
3. 2 minutes: live journey from section 2.5 (register, author, draft, publish, search, comment, admin delete).
4. 40 seconds: one failure case (for example, a reader gets 403).
5. 30 seconds: run the tests and show the result.
6. 20 seconds: known gaps and next steps.

17. Final checklist for the Blog project

#	Deliverable	Done?
1	README.md	
2	architecture.pdf and Lucidchart editable link	
3	database-design.pdf, ERD link, schema.sql	
4	api-workflows.pdf, openapi.yaml or api-spec.md	
5	test-cases.xlsx with actual results and evidence	
6	engineering-decisions.md	
7	GitHub repo and 3 to 5 minute demo link	
8	cost-and-scaling.xlsx with sources and dates	
9	sources-and-disclosure.md	
10	Blog entry in submission-index.md; Drive and repo access given to reviewer	

18. Suggested time plan for this project

Because there are 3 projects in one weekend, give the Blog project about one third of your time.

Block	Work
1	Requirements, ERD, architecture and API plan (diagrams in Lucidchart)
2	Backend: auth, posts, categories, comments, search
3	Frontend pages and connection to the API
4	Tests and fixing bugs
5	Cost sheet, scaling, decisions, disclosure, README, demo video

Tip: Build the Blog project in a clean, reusable way. Auth, roles and Docker setup can be reused in the E-commerce and Marketplace projects, which saves a lot of time (if you reuse your own code, mention it in the disclosure file).

19. Interview preparation

Be ready to answer these without notes:

- Why Django and Next.js? Why JWT? Where do you store the token and what is the risk?
- Show me where the permission check for editing a post is written.
- What happens if two users publish or comment at the same time?
- Why this index? What happens to search speed without it?
- How does your slug stay unique?
- What would break first at 10,000 users and why?
- Which part is not implemented and why?
- Make a small change live: for example, add a field to posts or change a permission rule.

20. Next step

In the next message you can ask for any one part and we will do it in detail, for example: the ERD and schema.sql, the Django models and project setup, the API spec, the test case sheet, or the cost sheet structure. Tell me which one to start with.